Algorithms: Homework 4

Yifan Chen

2026 Fall

Problem 5

(Source: KT book pp. 200, Problem 23.)

Recall the problem of computing a minimum-cost arborescence in a directed graph $G = (V, E)$, with a cost $c_e \geq 0$ on each edge. Here we will consider the case in which G is a directed acyclic graph—that is, it contains no directed cycles.

As in general directed graphs, there can be many distinct minimum-cost solutions. Suppose we are given a directed acyclic graph $G = (V, E)$, and an arborescence $A \subseteq E$ with the guarantee that for every $e \in A$, e belongs to some minimum-cost arborescence in G. Can we conclude that A itself must be a minimum-cost arborescence in G? Give a proof or a counterexample with explanation.

Answer:

This conclusion is true.

In a DAG, an arborescence A is min-cost if and only if the in-edge of each non-root vertex in A is one of the min-cost in-edges of itself. If we arbitrarily choose the min-cost in-edge for every non-root vertex, then, as we discussed in class, since every vertex except the root has exactly 1 in-degree, the picked edges form a graph whose every connected component is an out-directed tree (including the root) or an out-directed pseudotree (not including the root). However, there are no cycles in the DAG, so the picked edges form a connected out-directed tree.

Therefore, every edge $e \in A$ belonging to some min-cost arborescence in G is one of the min-cost in-edges of its destination, so A satisfies the above property, which implies A is a min-cost arborescence.

Problem 6

(Source: KT book pp. 205, Problem 32.)

Consider a directed graph $G = (V, E)$ with a root $r \in V$ and nonnegative costs on the edges. In this problem we consider variants of the minimum-cost arborescence algorithm.

(a) The algorithm discussed in Section 4.9 works as follows. We modify the costs, consider the subgraph of zero-cost edges, look for a directed cycle in this subgraph, and contract it (if one exists). Argue briefly that instead of looking for cycles, we can instead identify and contract strong components of this subgraph.

(b) In the course of the algorithm, we defined y_v to be the minimum cost of an edge entering v, and we modified the costs of all edges e entering node v to be $c'_e = c_e - y_v$. Suppose we instead use the following modified cost: $c''_e = \max(0, c_e - 2y_v)$. This new change is likely to turn more edges to 0 cost. Suppose now we find an arborescence T of 0 cost. Prove that this T has cost at most twice the cost of the minimum-cost arborescence in the original graph.

(c) Assume you do not find an arborescence of 0 cost. Contract all 0-cost strong components and recursively apply the same procedure on the resulting graph until an arborescence is found. Prove that this T has cost at most twice the cost of the minimum-cost arborescence in the original graph.

Answer:

(a) For part (a), any SCC must have an arborescence subgraph starting from any vertex (otherwise there will be some vertex that cannot be reached). Therefore, we only need to choose one in-edge of this SCC, which allows the SCC to be contracted as a vertex.

(b) For part (b), consider how the output could deviate from the answer. The only problem is we may undervalue an edge e with $0 < c'_e \leq y_v$ which has $c''_e = 0$, and cutting off this edge to pick another one can get a better answer. However, the undervalue is at most $c'_e \leq y_v$. Since each edge can be undervalued at most once, the total deviation is at most $\sum y_v \leq$ answer, which implies output $\leq$ answer $+ \sum y_v \leq 2 \cdot$ answer.

(c) For part (c), we only need to combine (a) and (b). In a contraction, we are doing the min-cost arborescence problem inside each SCC: the component is expanded back by an arborescence rooted at the vertex entered by its in-edge, which exists by (a) and costs 0 at this stage. In each 0-cost SCC, we can apply the arguments in (b) to verify this contraction deviates at most y_v at each v whose in-edge is settled in this stage, hence at most $\sum y_v$ in total for this stage. At every stage the y_v of the node containing a vertex v is at most the cost of any in-edge of that node at this stage, and the modification at that stage subtracts at least this y_v from that edge, so adding over all stages, the total undervaluation at v is at most the original cost of the in-edge that any fixed arborescence gives to v. That is, the total undervaluation is at most answer. Therefore, this T has cost at most twice the answer.

Problem 7

In this problem, we introduce a very useful and important concept in combinatorics, matroid. Matroids have very close connections with greedy algorithms and submodular functions (another important concept we will explore in later homeworks). The theory of matroids is a deep branch of combinatorics.

A finite matroid M is a pair $(E, \mathcal{I})$, where E is a finite set (called the "ground set") and $\mathcal{I}$ is a collection of subsets of E. Each element in $\mathcal{I}$ (which is a subset of E) is called an *independent set* (this notion has nothing to do with the notion of independent set in graph theory). Moreover, $(E, \mathcal{I})$ has to satisfy the following properties:

1. $\emptyset \in \mathcal{I}$.
2. Every subset of an independent set is independent, i.e., if $A \subset B$, $B \in \mathcal{I}$, then $A \in \mathcal{I}$.
3. If A and B are two independent sets of $\mathcal{I}$ and A has more elements than B, then there exists an element $e \in A - B$ such that $B \cup \{e\}$ is an independent set.

Try to answer the following questions.

1. Suppose E is a set of real vectors of the same length and

$$\mathcal{I} = \{S \subseteq E \mid \text{all vectors in } S \text{ are linearly independent}\}.$$

   Prove that $(E, \mathcal{I})$ is a matroid.

2. Suppose E is the set of edges of a given graph G and

$$\mathcal{I} = \{S \subseteq E \mid \text{all edges in } S \text{ constitute a forest of } G\}.$$

   (A forest is just a collection of trees. Note that a singleton node is also a tree.) Prove that $(E, \mathcal{I})$ is a matroid.

3. Suppose each element $e \in E$ is associated with a weight $c_e \geq 0$. Design a polynomial-time algorithm to find a max-weight independent set. In other words, the algorithm should output an independent set $A \in \mathcal{I}$ such that $\sum_{e \in A} c_e$ is maximized. You need to prove the correctness of your algorithm. (Hint: Kruskal's algorithm.)

(Sidenote: There are several important problems which also fit into the framework of matroid. We will cover some of them in the later part of this course. Matroid also has a deep connection with linear programming and other set systems (like polymatroid and greedoid). You will learn some of them later.)

Answer:

1.
   - $\emptyset$ is linearly independent.
   - If S is linearly independent, then any subset of S is linearly independent.
   - If A, B are both linearly independent sets, with $|A| > |B|$, then $\text{Span}(B)$ cannot contain all the $|A|$ linearly independent vectors. Since $\dim(\text{Span}(B)) = |B| < |A|$, there exists some $v \in A - B$ such that $B \cup \{v\}$ is linearly independent.

2.
   - If there are no edges, then each vertex is a connected component which is a tree.
   - If S forms a forest, then any subset of S forms a forest as well since there is no cycle.

Problem 7 continued on next page...

- If A, B form two forests, with $|A| > |B|$, consider each connected component V_i of the graph formed by B, there are at most $|V_i| - 1$ edges in A which link two vertices belonging to V_i. Since $|A| > |B| = \sum_i (|V_i| - 1)$, there must exist some $e \in A$ that links two vertices from different connected components, then e satisfies $e \in A - B$ and $B \cup \{e\}$ forms a forest.

3. Consider the Kruskal's algorithm: sort $e \in E$ in the decreasing order of c_e, and try adding e into an initially empty set S by the order, accept if and only if $S \cup \{e\}$ is still an independent set.

   According to property 3, the size of each max size independent set is the same. Assume this algorithm does not give the max-weight independent set X, but gives S. Let us write the elements in X and S in sorted order as $a_1, a_2, \ldots, a_k$ and $b_1, b_2, \ldots, b_k$. Consider the first position that $a_i > b_i$, then we have $\{b_{1\ldots i-1}\}$ is an independent set with size less than $\{a_{1\ldots i}\}$, thus we can find $1 \le p \le i$ such that $a_p \notin S$ and $\{b_{1\ldots i-1}\} \cup \{a_p\}$ is an independent set, which contradicts the algorithm since $a_p > b_i$ is acceptable but not accepted.

   Therefore, no such X exists, and our algorithm is optimal.

AI Disclosure

I used an AI assistant on all problems in this homework, for grammar polishing, LaTeX typesetting, and checking the correctness of my arguments. All mathematical ideas and proofs are my own.